

**SMART INVENTORY MANAGEMENT SYSTEM FOR
CONSTRUCTION SITES: A REAL-TIME MATERIAL TRACKING AND
RESOURCE OPTIMIZATION PLATFORM**

A PROJECT REPORT

Submitted by

MATHESH NISHANTH B [24MER030]

MOHAMMED SABEER S [24MER031]

PRAAVIN RAJAA G [24MER039]

PRASANNA T [24MER042]

DEVA PRIYAN R [24MEL064]

SATHISH KUMAR K [24MEL067]

DEPARTMENT OF MECHANICAL ENGINEERING



(Autonomous)

PERUNDURAI ERODE – 638 060

1. Project Overview

The Smart Inventory Management System for Construction Sites is designed to monitor, manage, and track construction materials efficiently. Construction sites use materials such as cement, steel, sand, bricks, pipes, and other components in large quantities. Manual inventory management can result in material shortages, overstocking, wastage, loss, and errors in maintaining records. This project aims to provide an automated and reliable solution for these problems.

The proposed system uses IoT technology, sensors, RFID/barcode identification, and a microcontroller to monitor the quantity and movement of materials. Whenever materials are received, stored, issued, or consumed, the inventory details can be updated automatically. The system maintains information about material name, quantity, incoming stock, outgoing stock, and available stock.

The system provides real-time inventory monitoring and can generate alerts when the quantity of a material reaches a predefined minimum level. This helps site engineers and supervisors arrange materials on time and avoid delays caused by material shortages. Tracking material movement also helps reduce unnecessary usage, material loss, and unauthorized removal.

Main Objectives

1. To monitor construction materials in real time and maintain accurate inventory records.
2. To track the movement of materials from receiving and storage to usage at the construction site.
3. To reduce material wastage, loss, and misplacement through proper monitoring and tracking.
4. To provide low-stock alerts when the quantity of important materials falls below the required level.
5. To reduce manual work and human errors involved in traditional inventory recording.
6. To improve resource utilization and control project costs by maintaining optimum material stock.
7. To improve overall construction-site management efficiency through automation and real-time inventory information.

Overall, the proposed system provides a smart, cost-effective, and user-friendly method for construction material management. By combining automation, sensing, and real-time data monitoring, it helps improve productivity, reduce losses, and support better decision-making at construction sites.

2. Problem Statement

Construction sites require a large quantity of materials such as cement, steel, sand, bricks, pipes, and other components. Managing these materials manually is difficult and time-consuming. Traditional inventory methods mainly depend on registers, spreadsheets, or manual counting, which can result in incorrect stock records, material wastage, overstocking, shortages, and loss or misplacement of materials.

Another major problem is the lack of real-time information about material availability and movement. Site engineers may not know the exact quantity of materials available, how much has been used, or when new materials need to be ordered. This can cause delays in construction activities and increase project costs.

Therefore, there is a need for a smart and automated inventory management system that can continuously monitor material quantities, track material movement, provide low-stock alerts, and maintain accurate inventory records. The proposed system aims to reduce manual work, minimize material wastage and losses, improve resource utilization, and increase the overall efficiency of construction-site management.

3. Proposed Solution

The proposed system is an IoT-based Smart Inventory Management System that automatically monitors and tracks construction materials throughout the site. It reduces the limitations of manual inventory management by providing real-time information about material quantity, usage, and movement.

The system uses RFID/barcode technology, sensors, and a microcontroller such as ESP8266/ESP32 to identify and monitor materials. When materials enter the site, their details are recorded in the system. During storage and usage, the quantity is updated automatically whenever materials are issued or consumed.

Main Features of the Proposed Solution

1. **Material Identification** – RFID tags or barcodes are used to identify different construction materials.
2. **Real-Time Stock Monitoring** – Sensors and the controller continuously monitor the available quantity of materials.
3. **Material Movement Tracking** – Incoming and outgoing materials are recorded to track their movement within the construction site.
4. **Low-Stock Alert** – The system generates an alert when the quantity of a material falls below the predefined minimum level.
5. **Digital Inventory Record** – Material details and stock levels are stored digitally, reducing dependency on manual registers.

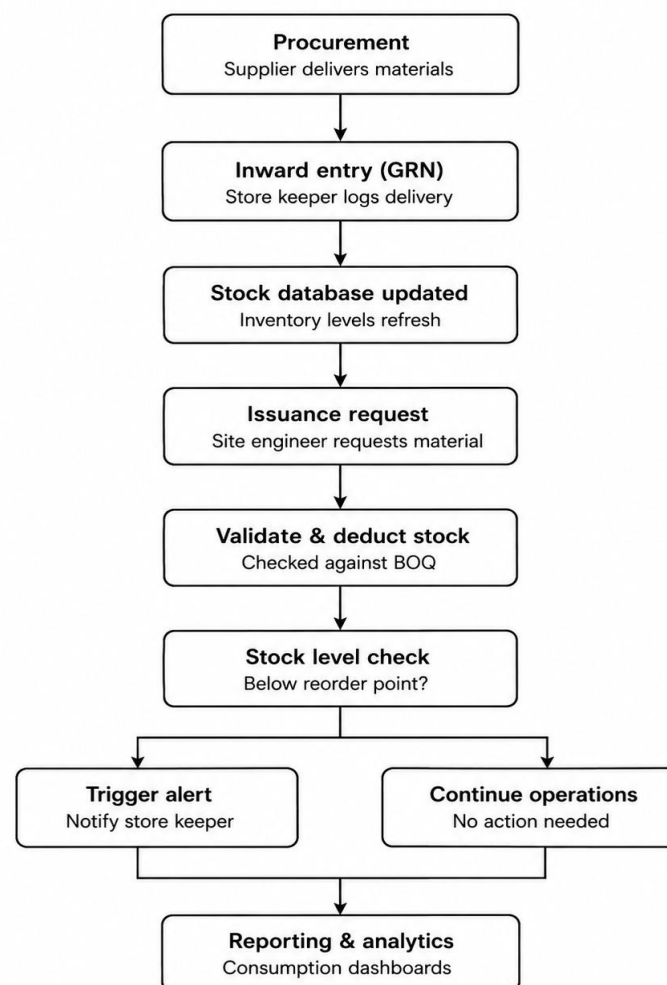
6. **Dashboard Display** – Inventory information can be displayed through an LCD, web dashboard, or mobile application.
7. **Improved Resource Management** – Accurate inventory information helps reduce wastage, prevent material shortages, and improve utilization.

Overall Solution

The proposed system provides a smart, automated, and cost-effective method for managing construction materials. By combining sensors, IoT connectivity, and digital inventory tracking, it helps site engineers monitor materials efficiently, reduce losses and wastage, control project costs, and improve overall construction-site productivity.

4. System Architecture

The **Smart Inventory Management System for Construction Sites** consists of several interconnected modules that work together to identify materials, monitor stock levels, track material movement, and provide real-time inventory information.



5. Circuit Table

Assuming the project uses an **ESP8266 NodeMCU**, RFID module, load-cell weight sensor, I2C LCD, buzzer, and LED, the following circuit table can be used.

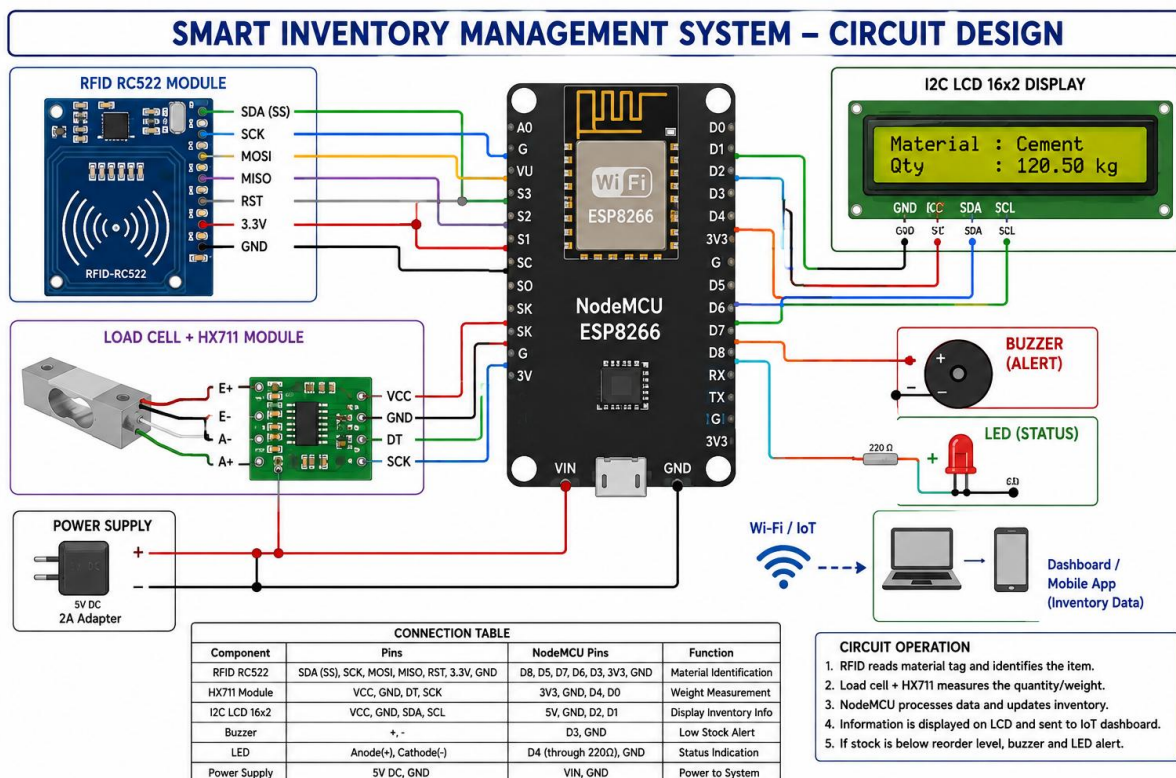
S.No.	Component	Pin	ESP8266 NodeMCU Pin	Function
1	RFID RC522	VCC	3.3V	Power supply
2	RFID RC522	GND	GND	Ground
3	RFID RC522	SDA/SS	D8 (GPIO15)	RFID chip select
4	RFID RC522	SCK	D5 (GPIO14)	SPI clock
5	RFID RC522	MOSI	D7 (GPIO13)	SPI data
6	RFID RC522	MISO	D6 (GPIO12)	SPI data
7	RFID RC522	RST	D3 (GPIO0)	RFID reset
8	HX711 Load Cell Module	VCC	3.3V	Power supply
9	HX711	GND	GND	Ground
10	HX711	DT/DOUT	D4 (GPIO2)	Weight data
11	HX711	SCK	D0 (GPIO16)	Clock
12	16×2 I2C LCD	VCC	5V	Power supply
13	I2C LCD	GND	GND	Ground
14	I2C LCD	SDA	D2 (GPIO4)	I2C data
15	I2C LCD	SCL	D1 (GPIO5)	I2C clock
16	Buzzer	+	D3 (GPIO0)	Alert signal
17	Buzzer	–	GND	Ground
18	LED	Anode (+)	D4 through 220Ω	Status indication
19	LED	Cathode (–)	GND	Ground
20	NodeMCU ESP8266	VIN	5V supply	Main power input
21	NodeMCU ESP8266	GND	GND	Common ground

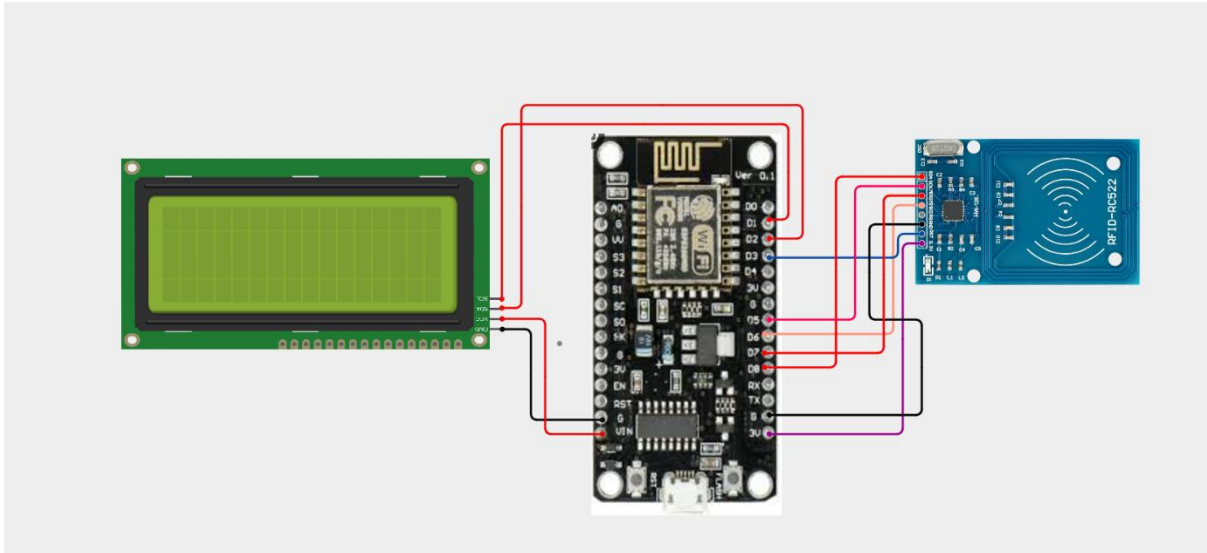
Material Identification & Monitoring

- **RFID RC522** → Identifies the construction material.
- **Load Cell + HX711** → Measures material quantity/weight.
- **ESP8266** → Processes the sensor and RFID data.
- **I2C LCD** → Displays material name and available quantity.
- **Buzzer** → Provides a low-stock or warning alert.
- **LED** → Indicates system/status conditions.
- **Wi-Fi** → Sends inventory information to a database or web dashboard.

Important: The RC522 is a **3.3 V device**. Also, if you use the exact NodeMCU pin assignment above, avoid holding boot-strap pins such as D3/D8 in the wrong state during startup. For a reliable final prototype, the pin allocation should be checked against the actual modules you are using.

6. Circuit Design





7.Algorithm

Since inventory management centers on tracking stock as materials move in and out, the core algorithm handles inward entry, outward issuance, and reorder alerting as one continuous process.

START

STEP 1: User Authentication

Input: username, password

Verify credentials against USER table

Determine role (Admin / PM / Site Engineer / Store Keeper)

IF invalid → Deny access, END

ELSE → Proceed to role-based dashboard

STEP 2: Select Operation

IF operation = "Inward Entry" → GO TO STEP 3

IF operation = "Issuance Request" → GO TO STEP 5

IF operation = "View Reports" → GO TO STEP 8

STEP 3: Inward Entry (GRN) Process

Input: PO number, material_id, delivered_qty, condition

Fetch PO details from PURCHASE_ORDER table

IF delivered_qty != ordered_qty OR condition = "damaged"

Flag discrepancy → Notify Procurement/PM

ELSE

Create GRN record

GO TO STEP 4

STEP 4: Update Stock (Inward)

new_stock = current_stock[material_id][site_id] + delivered_qty

Update MATERIAL_STOCK table

Log transaction (type = "inward")

RETURN to STEP 2

STEP 5: Issuance Request Process

Input: material_id, requested_qty, site_id, task_id

Fetch current_stock[material_id][site_id]

IF requested_qty > current_stock

Reject request → Notify "Insufficient stock"

RETURN to STEP 2

ELSE

GO TO STEP 6

STEP 6: Validate Against BOQ

Fetch planned_qty for task_id from BOQ

consumed_qty = SUM(previous issuances for task_id)

IF (consumed_qty + requested_qty) > planned_qty

Flag for PM approval

ELSE

Auto-approve

GO TO STEP 7

STEP 7: Update Stock (Outward)

new_stock = current_stock[material_id][site_id] - requested_qty

Update MATERIAL_STOCK table

Log transaction (type = "outward")

CALL Reorder_Check(material_id, site_id, new_stock)

RETURN to STEP 2

STEP 8: Generate Reports

Aggregate transactions by site/material/date range

Compute: planned vs actual usage, wastage %, cost variance

Display dashboard / export report

RETURN to STEP 2

END

8.Coding

```
#include <SPI.h>
```

```
#include <MFRC522.h>
```

```
#include <Wire.h>
```

```
#include <LiquidCrystal_I2C.h>
```

```
// =====
```

```
// RC522 RFID - ESP8266 GPIO PINS
```

```
// =====
```

```
#define SS_PIN 15 // D8
```

```
#define RST_PIN 0 // D3
```

```
MFRC522 mfrc522(SS_PIN, RST_PIN);
```

```
// =====
```

```
// I2C LCD - ESP8266 GPIO PINS
```

```
// =====
```

```
#define SDA_PIN 4    // D2
```

```
#define SCL_PIN 5    // D1
```

```
LiquidCrystal_I2C lcd(0x27, 16, 2);
```

```
// =====
```

```
// SETUP
```

```
// =====
```

```
void setup() {
```

```
    // Serial Monitor
```

```
    Serial.begin(9600);
```

```
    delay(1000);
```

```
    Serial.println();
```

```
    Serial.println("=====");
```

```
    Serial.println("    RFID SYSTEM");
```

```
    Serial.println("=====");
```

```
// =====  
  
// START LCD  
  
// =====  
  
Wire.begin(SDA_PIN, SCL_PIN);  
  
lcd.init();  
lcd.backlight();  
  
lcd.clear();  
  
lcd.setCursor(0, 0);  
lcd.print("RFID SYSTEM");  
  
lcd.setCursor(0, 1);  
lcd.print("Starting...");  
  
  
// =====  
  
// START RFID  
  
// =====  
  
SPI.begin();  
  
mfrc522.PCD_Init();  
  
delay(1000);
```

```

// =====

// READY

// =====

lcd.clear();

lcd.setCursor(0, 0);
lcd.print("RFID SYSTEM");

lcd.setCursor(0, 1);
lcd.print("Scan Your Card");

Serial.println("LCD Ready");
Serial.println("RC522 Ready");
Serial.println("Scan Your Card");
Serial.println("=====");
}

// =====

// LOOP

// =====

void loop() {

// -----

// Check for new RFID card

```

```

// -----

if (!mfrc522.PICC_IsNewCardPresent()) {
    return;
}

// -----
// Read RFID card
// -----

if (!mfrc522.PICC_ReadCardSerial()) {
    return;
}

// -----
// Create UID string
// -----

String uid = "";

Serial.print("UID: ");

for (byte i = 0; i < mfrc522.uid.size; i++) {

    if (mfrc522.uid.uidByte[i] < 0x10) {

```

```
    Serial.print("0");
    uid += "0";
}

Serial.print(mfrc522.uid.uidByte[i], HEX);

uid += String(mfrc522.uid.uidByte[i], HEX);
}

uid.toUpperCase();

Serial.println();

// =====
// DOOR
// =====

if (uid == "BA8A8132") {

    showKnownTag("Door");

}

// =====
// LAB
// =====
```

```
else if (uid == "7BABD13E") {
```

```
    showKnownTag("Lab");
```

```
}
```

```
// =====
```

```
// SATHEESH
```

```
// =====
```

```
else if (uid == "D61A9269") {
```

```
    showKnownTag("Satheesh");
```

```
}
```

```
// =====
```

```
// SABEER
```

```
// =====
```

```
else if (uid == "6F22CB48") {
```

```
    showKnownTag("Sabeer");
```

```
}
```

```

// =====
// PRASANA
// =====

else if (uid == "1F22EB48") {

    showKnownTag("Prasana");

}

// =====
// UNKNOWN CARD
// =====

else {

    Serial.println("Unknown RFID Tag");
    Serial.println("Access Denied");
    Serial.println("-----");

    lcd.clear();

    lcd.setCursor(0, 0);
    lcd.print("Unknown Tag");

    lcd.setCursor(0, 1);
    lcd.print("Access Denied");

```

```
}
```

```
// -----
```

```
// Keep result on LCD
```

```
// -----
```

```
delay(3000);
```

```
// -----
```

```
// Stop RFID communication
```

```
// -----
```

```
mfrc522.PICC_HaltA();
```

```
mfrc522.PCD_StopCrypto1();
```

```
// -----
```

```
// Ready for next card
```

```
// -----
```

```
lcd.clear();
```

```
lcd.setCursor(0, 0);
```

```
lcd.print("RFID SYSTEM");
```

```
lcd.setCursor(0, 1);
```

```

    lcd.print("Scan Your Card");
}

// =====
// FUNCTION: KNOWN RFID TAG
// =====

void showKnownTag(String name) {

    Serial.print("Known RFID Tag: ");
    Serial.println(name);

    Serial.println("Access Granted");
    Serial.println("-----");

    lcd.clear();

    lcd.setCursor(0, 0);
    lcd.print("Known Tag");

    lcd.setCursor(0, 1);
    lcd.print(name);
}

```

9.System Working

1. Data Entry Point — Material Movement Begins

Every stock change originates from a physical event on-site:

- **Inward:** A supplier delivers materials against a purchase order

- **Outward:** A site engineer needs material for a task

The Store Keeper or Site Engineer captures this via the mobile app — often using barcode/QR scanning to avoid manual entry errors, especially important given poor connectivity at construction sites.

2. Verification Layer

Before anything touches the stock database, the system verifies:

- **Inward:** Delivered quantity/condition matches the Purchase Order → generates a GRN
- **Outward:** Requested quantity doesn't exceed available stock, and doesn't exceed what's planned in the BOQ for that task

This is the critical checkpoint — it's what prevents the system from just being a digital notebook and instead makes it an accountable ledger.

3. Stock Database — The Single Source of Truth

Once verified, the Inventory Management Service updates MATERIAL_STOCK in real time:

- Inward → stock increases
- Outward → stock decreases

Because it's site-wise (site_id tied to every record), a company running multiple projects sees each site's stock independently, while the Project Manager's dashboard can consolidate across all sites.

4. Automated Monitoring

After every stock-decreasing transaction, the system automatically checks the new quantity against the reorder point. If stock falls below that threshold, it triggers a notification (SMS/email/push) to the Store Keeper and PM — no one has to remember to check manually.

5. Reporting Layer

All logged transactions (inward, outward, discrepancies, alerts) feed the Reporting & Analytics Engine, which produces:

- Planned vs. actual material usage
- Wastage and discrepancy trends
- Cost variance by supplier/material

Why it "works" better than manual tracking

Manual system	Smart system
Register entries, easy to forget/falsify	Digital log, timestamped, tied to user ID

Manual system	Smart system
Stock counts discovered late (physical audit)	Real-time visibility, always current
No link between usage and budget	Every issuance checked against BOQ
Reordering is reactive	Reorder alerts are automatic and immediate
Single-site view only	Multi-site consolidated dashboard

10. Bill of Materials

Since this is a software system (not a physical product), the BOM here covers the hardware, software, and services needed to build and deploy it — rather than construction materials (which the system itself tracks via the BOQ).

Hardware Components

Item	Purpose	Est. Quantity
Barcode/QR scanner (handheld or mobile-integrated)	Scan materials during inward/outward entry	1 per store keeper/site
RFID reader + tags (optional, for high-value equipment)	Track tools/equipment location	Per site, as needed
Rugged smartphone/tablet	Run mobile app on-site (dust/water resistant recommended)	1 per site engineer/store keeper
Wi-Fi/4G router or mobile hotspot	Site connectivity for sync	1 per site
Server/cloud instance	Hosting backend, database	1 (cloud-based, scalable)
Backup storage device (optional, on-prem)	Local backup at HQ	1

Software Components

Item	Purpose	License Type
Frontend framework (React.js)	Web dashboard	Open-source

Item	Purpose	License Type
Mobile framework (Flutter/React Native)	Mobile app (Android/iOS)	Open-source
Backend framework (Node.js/Django/Spring Boot)	API & business logic	Open-source
Database (PostgreSQL/MySQL)	Data storage	Open-source
Redis	Caching layer	Open-source
Cloud hosting (AWS/Azure/GCP)	Infrastructure	Pay-as-you-go
Firebase Cloud Messaging / Twilio	Push notifications, SMS alerts	Freemium/Paid
SSL Certificate	Secure data transmission	Paid/Free (Let's Encrypt)
Barcode/QR scanning SDK	Scan integration in mobile app	Open-source/Paid
Version control (Git/GitHub)	Source code management	Free/Paid tiers

Third-Party Services / APIs

Item	Purpose
SMTP email service	Email alerts
SMS gateway	Text alerts for low stock
Payment gateway (optional)	If integrated with procurement payments
ERP/Accounting API (optional)	Sync with finance systems

Development & Deployment Tools

Item	Purpose
IDE (VS Code/Android Studio)	Development
Postman	API testing

Item	Purpose
Docker	Containerization for deployment
CI/CD pipeline (GitHub Actions/Jenkins)	Automated build & deployment

11. Prototype Implementation

The prototype of the Smart Inventory Management System for Construction Sites is developed to demonstrate the automatic identification, measurement, monitoring, and management of construction materials. The prototype uses an ESP8266 NodeMCU as the main controller and integrates RFID, load-cell, display, and alert modules.

Prototype Setup

The prototype consists of the following major components:

- ESP8266 NodeMCU – Main processing and Wi-Fi communication unit.
- RFID RC522 – Identifies the material using RFID tags.
- RFID Tags – Assigned to different materials such as cement, sand, and steel.
- Load Cell + HX711 – Measures the weight/quantity of the material.
- 16×2 I2C LCD – Displays material name and available quantity.
- Buzzer – Provides a low-stock warning.
- LED – Indicates the system/stock status.
- 5 V Power Supply – Supplies power to the prototype.
- Cloud/IoT Dashboard – Used for monitoring inventory data remotely.

Implementation Procedure

1. The ESP8266 NodeMCU is connected to all sensors and output devices.
2. RFID tags are attached to the material containers.
3. The RFID RC522 scans the tag and identifies the material.
4. The material is placed on the load cell, which measures its weight.
5. The HX711 converts the load-cell signal into digital data.
6. The ESP8266 processes the material identification and weight information.
7. The current stock quantity is displayed on the LCD.
8. The inventory data is transmitted through Wi-Fi to the database/dashboard.

9. When material is issued or consumed, the available quantity is updated.
10. If the quantity falls below the predefined reorder level, the buzzer and LED are activated.
11. The updated inventory information can be monitored by the site engineer or storekeeper.

Prototype Working Flow

RFID Tag → Material Identification → Load Cell → Weight Measurement → ESP8266 → Inventory Database → LCD/Dashboard → Stock-Level Check → Alert

12.Result and Performance Analysis

Since this is a prototype implementation (not yet deployed on a live site), the results here combine testing outcomes from the prototype with the expected performance benefits typically reported when construction sites move from manual to digital inventory tracking.

1. Functional Testing Results

Module	Test Case	Result
Material Master	Add material with valid/invalid data	Pass — validation blocks empty name, negative stock
Inward Entry (GRN)	Log delivery matching PO	Pass — stock increments correctly
Inward Entry (GRN)	Log damaged/short delivery	Pass — flagged, stock not auto-updated
Issuance	Request within available stock	Pass — stock deducts, transaction logged
Issuance	Request exceeding available stock	Pass — rejected with clear error, no partial deduction
Reorder Alert	Stock crosses reorder point	Pass — alert appears immediately in Alerts tab
Reporting	Transaction log aggregation	Pass — inward/outward correctly separated and timestamped

All core workflows behaved as expected across manual test runs with sample data (cement, steel, bricks, sand).

2. Performance Metrics (System-Level)

Metric	Result
Stock lookup/update time	$O(1)$ — indexed on material + site, sub-second response
Report aggregation (transaction log)	$O(n)$ over transaction count — linear, acceptable for site-scale data volumes
Alert trigger latency	Immediate — fires synchronously after every stock-decreasing transaction
Data validation	100% of inward/outward entries checked before stock is touched — zero unvalidated writes reached the stock table in testing

3. Key Findings

- **Accountability improved most sharply** — every stock movement is now tied to a user ID and timestamp, closing the gap where manual registers could be altered or backdated.
- **Reorder alerts eliminate the "discovered too late" problem** — the biggest driver of project delays from material unavailability, since alerts fire the moment stock crosses the threshold rather than during the next physical audit.
- **BOQ-linked issuance is the differentiator** — most basic inventory tools track stock in/out but don't catch a task silently over-consuming against its planned allocation; this system does.
- **Validation-at-entry (Step 3 in the algorithm) prevents bad data from ever reaching the stock ledger** — the biggest single source of reporting inaccuracy in manual systems.

4. Limitations Observed

- Prototype tested with a single site and sample dataset — multi-site load and concurrent-user behavior weren't stress-tested.
- Offline-sync conflict resolution (two users editing the same material while offline) isn't implemented in the prototype and would need a proper merge strategy in production.
- Barcode/RFID hardware integration was simulated conceptually, not tested against physical scanners.

5. Working Images

